

Vehicle-mounted Balance Ball Motion Control System

Vehicle-mounted Balance Ball Motion Control System

- 1. Hardware Platform
- 2. Software Environment
- 3. Structure Description
 - 3.1 Component List and Material Preparation
 - 3.2 Physical Appearance
 - 3.3 Assembly Instructions
- 4. Brief Principles
 - 4.1 Wiring Pinout
 - 4.2 Physical Wiring Diagram
 - 4.3 Control Principle
- 5. Project File Structure
- 6. Core Parameter Configuration
 - 6.1 Servo Ball Control PID (app_k230.c)
 - 6.2 Line Tracking PID (Four_linewalking_debug.c)
 - 6.3 Task Line Tracking Speeds (questions.c)
 - 6.4 Task Timeout Settings (questions.c)
- 7. Task Description
 - 7.1 State Machine Structure (questions.h)
 - 7.2 Task Detailed Description
- 8. Key Code Explanation
 - 8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)
 - 8.2 Line Tracking — Debug_LineWalking_Run() (Four_linewalking_debug.c)
 - 8.3 Line Tracking + Ball Control Combined — Debug_LineWalking_WithBall() (Four_linewalking_debug.c)
 - 8.4 Task 2 Steel Ball Reciprocation (questions.c)
- 9. Operation Procedure

1. Hardware Platform

Component	Model/Description
Main Controller	MSPM0G3507/MSPM0G3519 Development Board
Camera	K230 AI Vision Module (YOLOv8 Steel Ball Detection)
Line Tracking	4-Channel Red Line Tracking Module (IO Output)
Display	OLED 128×32 (I2C)
Servo	9G Servo (Steel Ball Tilt Control)
Motors	Two L-Type 520 Encoded Motors (I2C Driver Board)
Button	Expansion Board K1 Single Key Control

2. Software Environment

- **IDE:** Keil MDK V5 (ARM Compiler V6.7)
- **SDK:** TI MSPM0 SDK 2.02.00.05

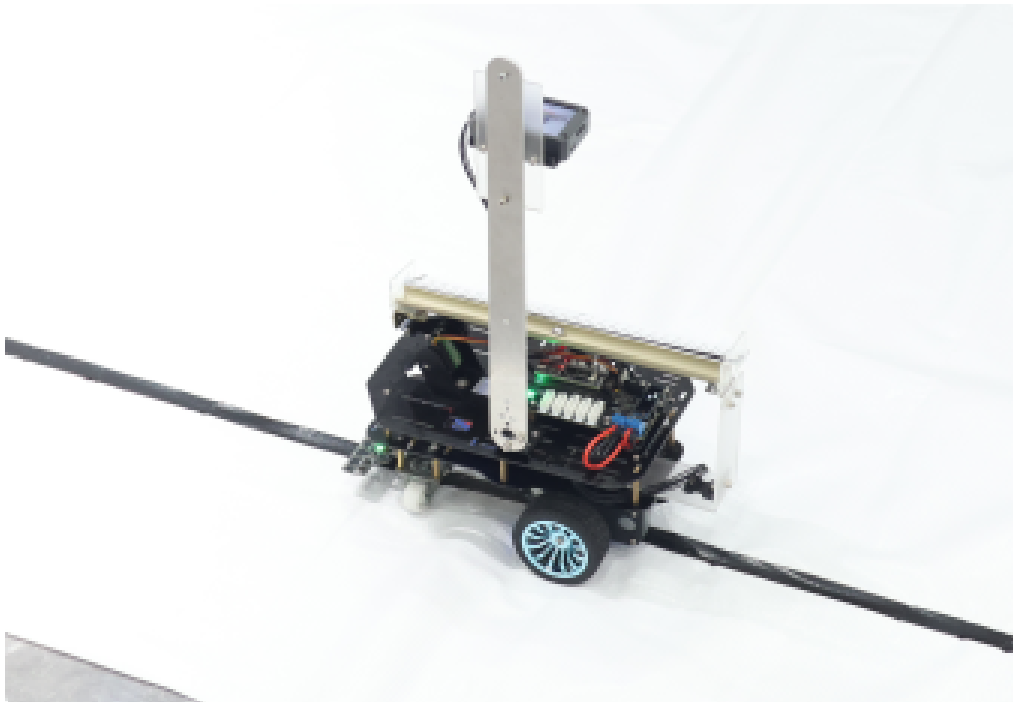
3. Structure Description

3.1 Component List and Material Preparation

- **Chassis Parts:** Black metal etched baseplate, drive motors, wheels. The chassis comes with reserved screw holes and wire routing holes.
- **Robotic Arm Structure:** Blue/black metal servo brackets, metal connecting rods, metal L-type/linear connectors, servos, metal flanges.
- **Horizontal Ruler Sliding Platform:** Transparent acrylic housing, hand-written digital ruler with zero reference line, internal transmission guide rails/screws.
- **Top Vision Bracket:** Brushed metal strips (aluminum alloy/stainless steel), transparent acrylic L-brackets, K230 module.
- **Hardware Accessories:** M2/M3组合 screws, nuts, washers, black cable ties, heat shrink tubing.

Note: In addition to the accessories included in the v2 chassis kit, 9g servo, K230, and line tracking modules can be purchased from our store. Other connecting rods, rulers, water pipes, and related hardware accessories need to be obtained by yourself.

3.2 Physical Appearance



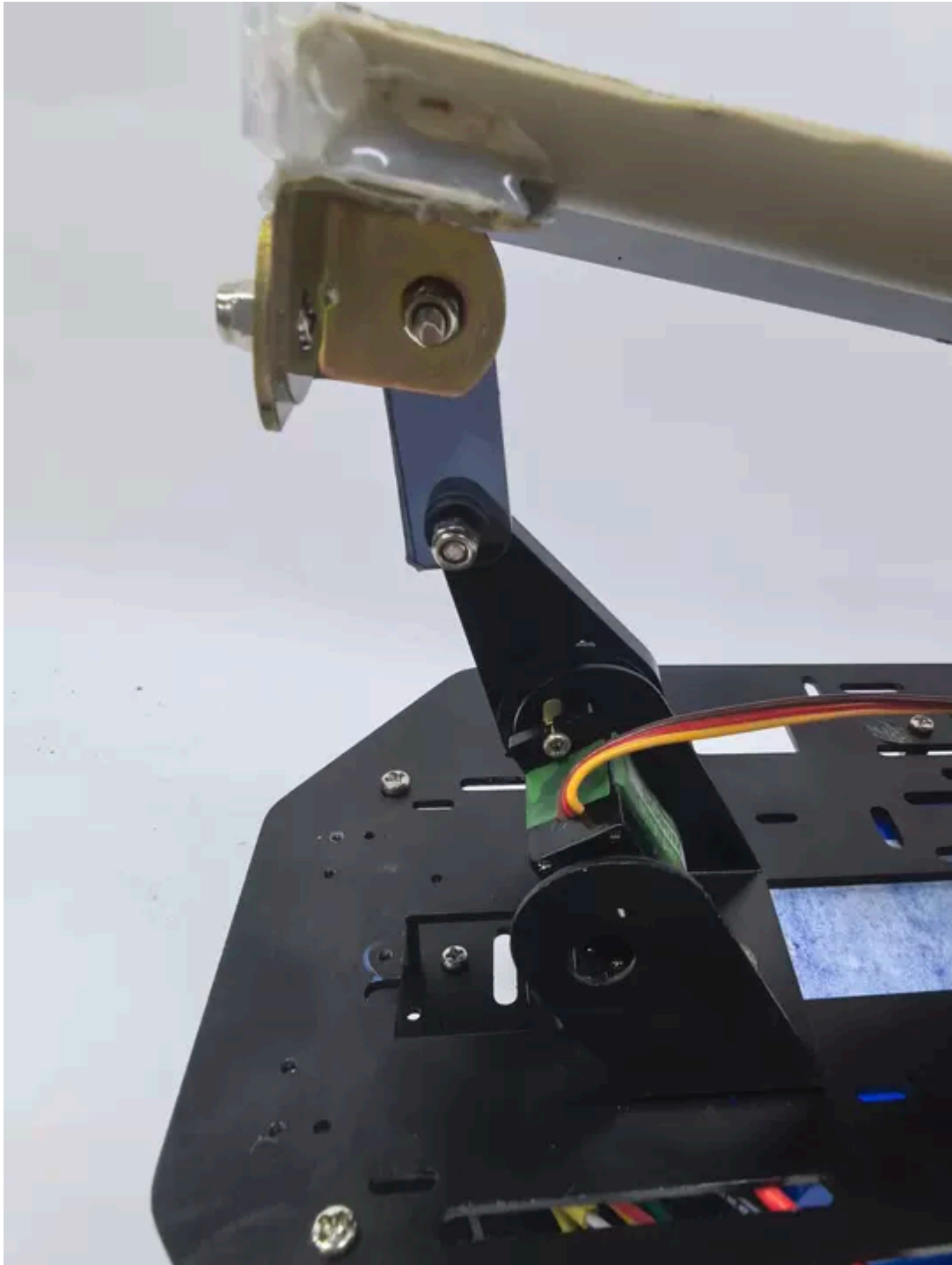












3.3 Assembly Instructions

Step 1: Install Mobile Chassis and Motors

- Fix the two drive motors (with wheels) on both sides of the black metal baseplate. Ensure the wheels face the same direction and make full contact with the ground.
- Install the main controller board MSPM0G and secure it with screws at the reserved position on the chassis. Note: Keep the main controller board away from the rotating space of wheels and motors.

Step 2: Build Horizontal Ruler and Translation Sliding Platform

- *Reference Figure 5:* Assemble the transparent acrylic housing and insert the cardboard with the scale

ruler inside. This structure is fixed at the front of the chassis.

- Note: The guide rails or screws inside the sliding platform need to remain horizontal, which is the key reference for subsequent robotic arm coordinate calibration.

Step 3: Assemble Robotic Arm Joints

- *Reference Figures 3 and 7:* Insert the green servo into the black metal bracket and lock it with matching screws.
- Install the servo arm and connect the blue/black metal connecting rods.
- Build the end effector (connector/claw) by referencing Figure 7. The image shows this component reinforced with a lot of **transparent acrylic and hot glue/resin**. During actual assembly, it is recommended to use 3D printed parts or metal parts instead to ensure sufficient structural strength.
- Fix the servo base at the rear of the chassis (refer to the layout position in Figure 7).

Step 4: Install Vertical Metal Bracket

- Reference Figures 1, 4, and 6: Take the brushed metal strip and install it vertically into the fixed slot at the center or front side of the chassis.
- When securing with screws, ensure the metal strip is perpendicular to the chassis plane and does not wobble. The nuts in Figure 6 are locked through from the back.

Step 5: Install Top Vision Sensor

- Reference Figure 35: Place the K230 module with its back facing outward. Use transparent acrylic L-brackets and M3 screws to fix the module at the top of the vertical metal bracket.
- Note: Adjust the sensor orientation to ensure it can see the track and ruler ahead.

4. Brief Principles

4.1 Wiring Pinout

Motor Wiring:

Motor	Driver Board
M2	M-
V	3V3
A	H1B
B	H1A
G	GND
M1	M+

4-Channel Motor Driver Board:

Motor Driver Board	MSPM0G
5V	5V

Motor Driver Board	MSPM0G
SCL	PA15
SDA	PA16
GND	GND

4-Channel Line Tracking Module:

Line Tracking Module	MSPM0G
5V	5V
X1	PA24
X2	PA25
X3	PA26
X4	PA27
GND	GND

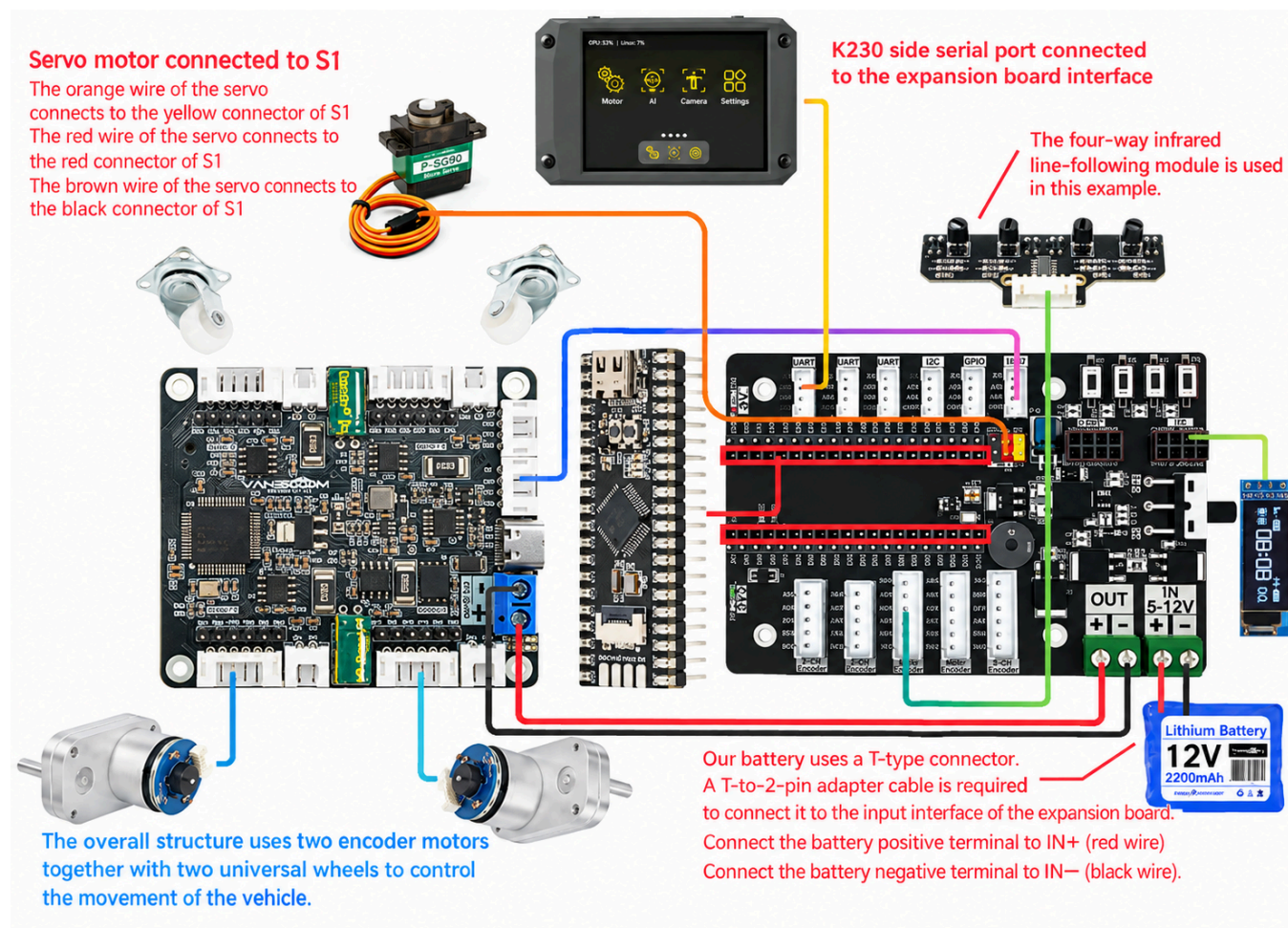
OLED:

OLED	MSPM0G
VCC	5V
GND	GND
SDA	PA28
SCL	PA31

K230 Camera:

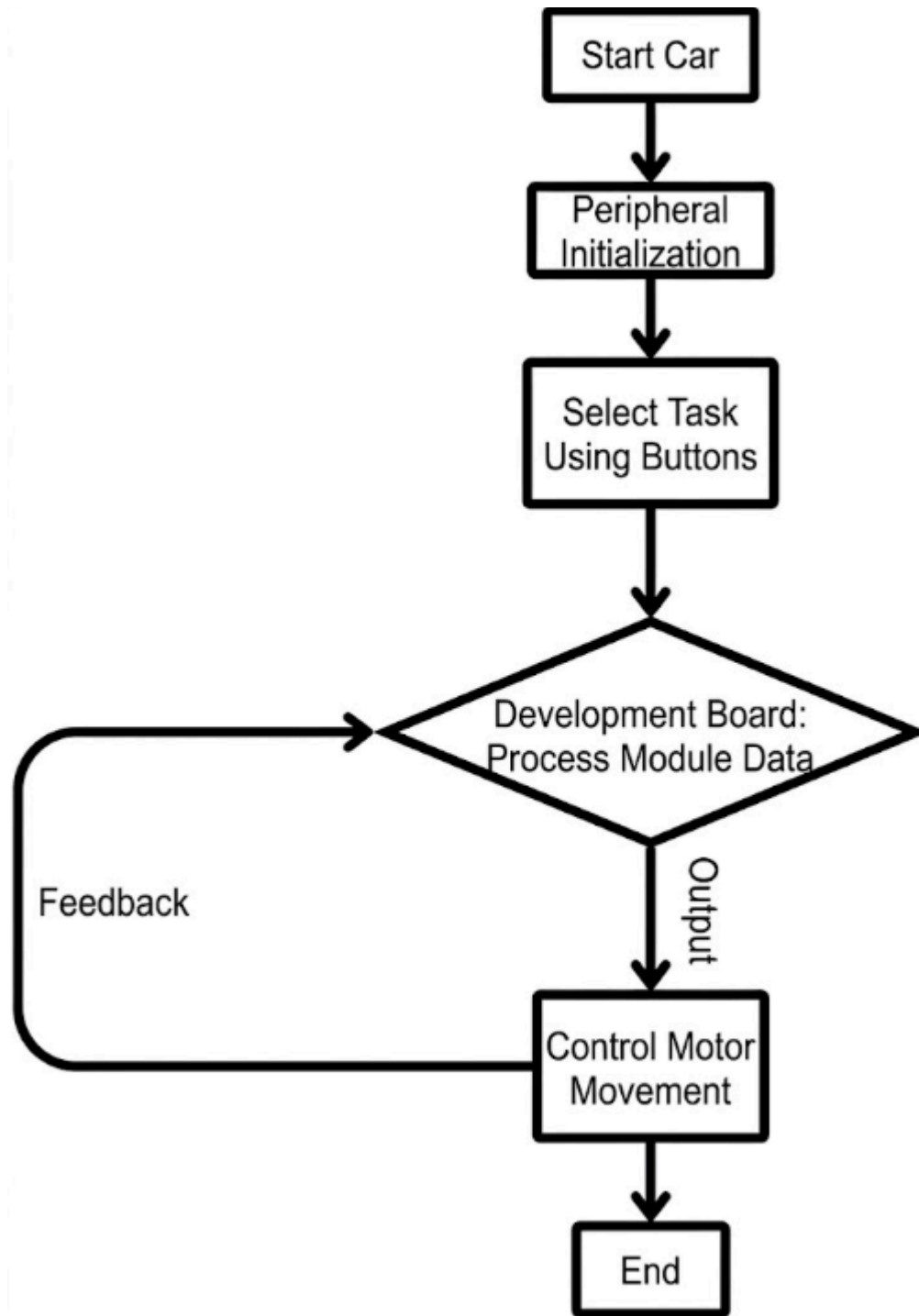
K230	MSPM0G
5V	5V
GND	GND
TXD	PB7
RXD	PB6

4.2 Physical Wiring Diagram



4.3 Control Principle

Car Line Tracking Control Flow:



1. Four-channel infrared sensor reads black line position -> Calculate deviation value
2. PID controller calculates turn correction -> Control motor differential steering
3. K230 camera detects steel ball position -> Servo PID controls pendulum angle
4. Timer timeout or waypoint detection -> Stop/Switch state

PID Control Parameter Description:

PID Parameter	Function	Adjustment Direction
P (Proportional)	Determines response speed; larger output when deviation is large	Too large: Oscillating snake pattern; Too small: Weak turning

PID Parameter	Function	Adjustment Direction
I (Integral)	Eliminates steady-state deviation; cumulative error correction	Too large: Overshoot; Too small: Cannot return from one side
D (Derivative)	Suppresses oscillation; early deceleration	Too large: Jitter; Too small: Cannot brake

5. Project File Structure

```
steel_ball_control_4walking/
├─ main.c                # Main program: peripheral initialization, main loop, state
machine dispatch
├─ APP/
│   ├─ app_k230.c/h      # K230 serial protocol + PID servo ball control (core)
│   ├─ app_pid.c/h       # PID controller (positional + incremental)
│   ├─ app_kalman.c/h    # 1D Kalman filter
│   └─ app_servo.c/h     # Servo PWM drive
├─ BSP/
│   ├─ Four_Tracking/
│   │   └─ Four_linewalking_debug.c/h # 4-channel line tracking + line tracking PID +
combined task (tracking + ball control)
│   │   └─ Four_linewalking.c/h# 4-channel line tracking module interface definition
│   └─ Motor/
│       ├─ app_motor.c/h    # Car motion control (Motion_Car_Control)
│       └─ bsp_motor_iic.c/h# Motor driver board I2C protocol
│   └─ OLED/
│       ├─ oled.c/h        # OLED driver + display function OLED_time()
│       └─ oledfont.h      # Font data
│   └─ Key/
│       └─ key.c/h         # Button driver + mode selection state machine
│   └─ Questions/
│       └─ questions.c/h   # Competition task state machine (Task 1~5)
│   └─ Task/
│       └─ task.c/h        # Timer task scheduler (Scheduler_Run)
│   └─ Timer/
│       └─ timer.c/h       # 1ms system clock Get_Time()
│   └─ usart.c/h           # Serial initialization (UART0 debug, UART1 K230)
└─ ti_msp_dl_config.c/h   # TI driver library configuration (generated by SysConfig)
```

6. Core Parameter Configuration

6.1 Servo Ball Control PID (app_k230.c)

```
/* ===== PID Parameters (Positional PID) ===== */
#define PID_KP      0.09f    /* Proportional gain: main response force */
#define PID_KI      0.005f   /* Integral gain: extremely small, only to eliminate steady-
state offset */
```

```

#define PID_KD      2.86f    /* Derivative gain: main damping, suppresses overshoot and
oscillation */

#define K230_CENTER  420.0f   /* K230 image center X pixel */
#define K230_MIN     99.0f    /* K230 image left boundary X pixel */
#define K230_MAX     599.0f   /* K230 image right boundary X pixel */
#define FOV_CM       28.0f    /* Camera field of view width (cm) */
#define PIPE_HALF_CM 12.5f    /* Half pipe length (cm), half of 25cm pipe */
#define PX_PER_CM_LEFT 30.72f /* Left side (330-99)/12.5 */
#define PX_PER_CM_RIGHT 28.28f /* Right side (599-330)/12.5 */

#define PWM_CC_CENTER 60.0f   /* Servo CC center value (1500us) */
#define PWM_CC_MIN    20.0f   /* Servo CC minimum value (500us) */
#define PWM_CC_MAX    100.0f  /* Servo CC maximum value (2500us) */
#define PWM_CC_SCALE  0.80f   /* PID output -> CC scaling factor */

#define DEADBAND_PX   3.0f    /* Deadband (pixels), no action within this range (~1.5mm)
*/
#define RATE_LIMIT    6.0f    /* Single frame CC change rate limit, prevents servo jerking
*/
#define KALMAN_Q       1.0f    /* K230 coordinate filter: process noise (larger = faster
response) */
#define KALMAN_R       0.3f    /* K230 coordinate filter: measurement noise (larger =
smoother) */

```

Bidirectional px_per_cm Compensation (camera not perfectly centered):

Target ≥ 0 (right side): $\text{px_per_cm} = (599 - 360) / 12.5 = 19.12$
 Target < 0 (left side): $\text{px_per_cm} = (360 - 99) / 12.5 = 20.88$

PID Tuning Guide:

Phenomenon	What to Adjust	Direction	Description
Ball swings back and forth near center	↑ KD	Increase damping	Larger KD = earlier braking
Ball stops before reaching target	↑ KP	Increase thrust	Larger KP = pushes farther
Ball overshoots target and bounces far	↓ KP	Decrease thrust	
Ball stabilizes but stays to one side, won't return	↑ KI	Strengthen integral	Larger integral = faster centering
Servo jitters when ball is stationary	↑ DEADBAND	Relax deadband	

6.2 Line Tracking PID (Four_linewalking_debug.c)

```
#define DEBUG_LINE_KP      55.0f
#define DEBUG_LINE_KI      0.00f
#define DEBUG_LINE_KD      645.0f
```

Line Tracking Tuning Guide:

Phenomenon	What to Adjust	Direction
Insufficient turning / crossing line	↑ KP	Increase
Snake-like movement	↑ KD or ↓ KP	Damping/reduce thrust
Always biased to one side	↑ KI	Strengthen integral

Inertia Compensation (for combined tracking + ball control task):

```
#define BALL_INERTIA_FWD    -0.8f  // Forward inertia compensation (cm), larger value = ball
// pushed more forward
#define BALL_INERTIA_TURN  -0.8f  // Turn inertia compensation (cm)
#define BALL_START_COMP    -5.0f  // Start compensation (cm), push forward when car
// accelerates and ball lags
#define BALL_STOP_COMP      5.0f  // Stop compensation (cm), pull back when car stops and
// ball rushes forward
#define RAMP_TIME_MS       3000   // Start gentle acceleration time (ms), larger = gentler
// start
```

6.3 Task Line Tracking Speeds (questions.c)

```
#define TASK1_SPEED        316    // Task 1 pure tracking speed
#define TASK3_SPEED        300    // Task 3 A->B + ball control speed
#define TASK4_SPEED        232    // Task 4 one lap + ball control speed
#define TASK5_SPEED        232    // Task 5 specified position speed
#define RAMP_TIME_MS       3000   // Start gentle acceleration time
```

6.4 Task Timeout Settings (questions.c)

```
#define TASK1_MAX_MS       20000  // Task 1 pure tracking 20s timeout
#define TASK2_MAX_MS       5000   // Task 2 ball reciprocating 5s timeout
#define TASK3_MAX_MS       8000   // Task 3 A->B + ball control 8s timeout
#define TASK4_MAX_MS       30000  // Task 4 one lap + ball control 30s timeout
#define TASK5_MAX_MS       30000  // Task 5 specified position 30s timeout
```

7. Task Description

7.1 State Machine Structure (questions.h)

Main State (Main_State):

Macro	Value	Task Description
STOP_STATE	0	Stop
TASK_LINE_LAP	1	Task 1: Pure tracking one lap back to A, $\leq 20s$
TASK_BALL_CTRL	2	Task 2: Steel ball $O \rightarrow +5 \rightarrow 0 \rightarrow -5$ (stop), $< 5s$
TASK_AB_STAB	3	Task 3: $A \rightarrow B$ + steel ball stable at O, $\leq 8s$
TASK_LAP_STAB	4	Task 4: One lap back to A + steel ball stable at O, $\leq 30s$
TASK_LAP_POS	5	Task 5: One lap back to A + steel ball at specified position, $\leq 30s$

Sub State (Sub_State):

Macro	Value	Description
SUB_IDLE	0	Idle
SUB_INIT	1	Initialize (set speed, start timer)
SUB_RUNNING	2	Running
SUB_DONE	3	Done (stop car but keep ball control; timeout also enters this state)

7.2 Task Detailed Description

Task 1 (Pure Tracking):

- Line tracking only, no ball control
- Speed: TASK1_SPEED = 316 mm/s
- Pass through 4 waypoints (A/B/C/D) and stop at point A
- Auto stop after 20s timeout

Task 2 (Steel Ball Control):

- Car stays still, servo control only
- Process: $O \rightarrow +5cm$ (stable) $\rightarrow 0cm$ (stable) $\rightarrow -5cm$ (stable)
- Each phase waits for ball to approach target ($\leq 0.5cm$) for 30 consecutive frames before switching
- 5s timeout
- After task completion, ball stays at -5cm position

Task 3 ($A \rightarrow B$ + Ball Control):

- Line tracking + servo ball control run simultaneously

- Speed: TASK3_SPEED = 130 mm/s
- Start from point A, pass through 1 waypoint to reach point B
- Ball stays stable at center O during driving
- Stop after 8s timeout, maintain ball control after stopping

Task 4 (One Lap + Ball Control):

- Line tracking + servo ball control run simultaneously
- Speed: TASK4_SPEED = 222 mm/s
- Clockwise one lap, pass through 4 waypoints back to A
- Ball stays stable at center O during driving
- Stop after 30s timeout, maintain ball control after stopping

Task 5 (Specified Position + Ball Control):

- Same as Task 4, but ball target position can be specified (Target_Position)
- Speed: TASK5_SPEED = 222 mm/s
- Stop after 30s timeout, maintain ball control after stopping

8. Key Code Explanation

8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)

```
void K230_Servo_Control(float target_cm)
{
    // Parse k230 serial frame (UART3 interrupt auto receives data)
    int r; do { r = parse_one(g_buf, g_wr); } while (r > 0);
    if (!g_valid) return;

    // Coordinate conversion cm -> pixels (bidirectional px/cm compensation)
    float px_per_cm;
    if (target_cm >= 0.0f)
        px_per_cm = (K230_MAX - K230_CENTER) / PIPE_HALF_CM; // Right side
    else
        px_per_cm = (K230_CENTER - K230_MIN) / PIPE_HALF_CM; // Left side
    float target_px = K230_CENTER + target_cm * px_per_cm;

    // Deadband + positional PID + rate limit + PWM output
    float error = target_px - kalman_Filter(&kf_x, g_x);
    if (error > -DEADBAND_PX && error < DEADBAND_PX) return;
    float cc = PID_To_CC_Pos(&g_pid, x_flt);
    // ... rate limit + CC clamping ...
    DL_TimerA_setCaptureCompareValue(PWM_Servo_INST, (uint16_t)cc, DL_TIMER_CC_0_INDEX);
}
```

PID Formula (Positional, app_pid.c): $u = K_p * e + K_i * \text{Integral}(e) + K_d * (e - e_{\text{last}})$

8.2 Line Tracking — Debug_LineWalking_Run() (Four_linewalking_debug.c)

```
/* Independent 4-channel line tracking function. */
void Debug_LineWalking_Run(void)
{
    extern volatile uint32_t g_task_start_ms;
    uint32_t now = Get_Time();

    if ((s_debug_initied == 0U) || (g_task_start_ms != s_debug_last_task_start_ms))
    {
        s_debug_last_task_start_ms = g_task_start_ms;
        Debug_Line_Reset();
        s_debug_initied = 1U;
    }

    if ((now - s_debug_last_run_ms) < DEBUG_LINE_PERIOD_MS)
    {
        return;
    }
    s_debug_last_run_ms = now;

    float raw_error = Debug_Line_GetError();
    float line_error = raw_error;
    float turn_cmd = 0.0f;

    if (s_debug_line_lost != 0U)
    {
        if (s_debug_lost_start_ms == 0U)
        {
            s_debug_lost_start_ms = now;
        }
        uint32_t lost_ms = now - s_debug_lost_start_ms;

        if (lost_ms >= DEBUG_LINE_LOST_STOP_MS)
        {
            control_speed(0, 0, 0, 0);
            return;
        }

        if (lost_ms >= DEBUG_LINE_LOST_DEBOUNCE_MS)
        {
            float lost_turn = (lost_ms >= DEBUG_LINE_LOST_ESCALATE_MS)
                ? DEBUG_LINE_LOST_TURN_HARD
                : DEBUG_LINE_LOST_TURN_SOFT;
            float lost_dir = 0.0f;
            if (s_debug_last_direction > 0.0f) { lost_dir = 1.0f; }
            else if (s_debug_last_direction < 0.0f) { lost_dir = -1.0f; }
            else if (s_debug_last_turn_dir > 0.0f) { lost_dir = 1.0f; }
            else if (s_debug_last_turn_dir < 0.0f) { lost_dir = -1.0f; }
            else { lost_dir = 1.0f; }
            turn_cmd = lost_dir * lost_turn;
        }
    }
}
```

```

        s_debug_err_last = raw_error;
    }
    else
    {
        turn_cmd = Debug_Line_PID(raw_error);
    }
}
else
{
    s_debug_lost_start_ms = 0U;

#if DEBUG_LINE_USE_KALMAN
    line_error = Kalman_Filter(&s_debug_line_kalman, raw_error);
#endif

    turn_cmd = Debug_Line_PID(line_error);

    if (turn_cmd > 1.0f)    { s_debug_last_turn_dir = 1.0f; }
    else if (turn_cmd < -1.0f){ s_debug_last_turn_dir = -1.0f; }
}

/* Convert current debug turn command to 5000 scale for ball control inertia
compensation. */
pid_output_IRR = (int)(turn_cmd * (BALL_INERTIA_PID_MAX / DEBUG_LINE_OUT_MAX));

float base_speed = (float)g_line_speed;

if (base_speed <= 0.0f)
{
    base_speed = DEBUG_LINE_BASE_SPEED;
}

float right_cmd = base_speed + turn_cmd;
float left_cmd  = base_speed - turn_cmd;

right_cmd = Debug_Clamp(right_cmd,
                        DEBUG_LINE_MIN_MOTOR_CMD,
                        DEBUG_LINE_MAX_MOTOR_CMD);
left_cmd = Debug_Clamp(left_cmd,
                        DEBUG_LINE_MIN_MOTOR_CMD,
                        DEBUG_LINE_MAX_MOTOR_CMD);

control_speed(0, (int16_t)right_cmd, 0, (int16_t)left_cmd);

if ((now - s_debug_last_print_ms) >= DEBUG_LINE_PRINT_PERIOD_MS)
{
    uint32_t ps = DL_GPIO_readPins(Sensor_four_PORT,
        Sensor_four_X1_PIN | Sensor_four_X2_PIN |
        Sensor_four_X3_PIN | Sensor_four_X4_PIN);
    int s1 = (ps & Sensor_four_X1_PIN) ? 1 : 0;
    int s2 = (ps & Sensor_four_X2_PIN) ? 1 : 0;
    int s3 = (ps & Sensor_four_X3_PIN) ? 1 : 0;
    int s4 = (ps & Sensor_four_X4_PIN) ? 1 : 0;
}

```

```

    printf("IR=%d%d%d%d E=%.2f T=%.2f L=%d R=%d LOST=%d\r\n",
           s1, s2, s3, s4,
           (double)line_error,
           (double)turn_cmd,
           (int)left_cmd,
           (int)right_cmd,
           (int)s_debug_line_lost);

    s_debug_last_print_ms = now;
}
}

```

8.3 Line Tracking + Ball Control Combined — Debug_LineWalking_WithBall() (Four_linewalking_debug.c)

```

/* 4-channel line tracking + steel ball inertia compensation. */
void Debug_LineWalking_WithBall(float ball_target_cm)
{
    static uint32_t last_task_start_ms = 0xFFFFFFFFU;
    static uint8_t  ramp_ready = 0;
    static uint8_t  stop_ready = 0;
    static uint32_t stop_start_ms = 0;
    static int      saved_spd = 0;
    static int      stop_spd = 0;
    extern volatile uint32_t g_task_start_ms;
    float start_comp = 0.0f;
    float stop_comp  = 0.0f;

    /* When task restarts, record cruise speed and reset gentle start/stop states. */
    if (g_task_start_ms != last_task_start_ms)
    {
        last_task_start_ms = g_task_start_ms;
        saved_spd = g_line_speed;
        ramp_ready = 1;
        stop_ready = 0;
        stop_start_ms = 0;
        stop_spd = 0;
    }

    /* Stop phase: linearly decelerate from current speed to 0 over BALL_STOP_TIME_MS. */
    if (g_slow_stop)
    {
        if (!stop_ready)
        {
            stop_start_ms = Get_Time();
            stop_spd = g_line_speed;
            stop_ready = 1;
        }

        uint32_t st = Get_Time() - stop_start_ms;
        if (st >= BALL_STOP_TIME_MS)

```

```

{
    g_line_speed = 0;
    control_pwm(0, 0, 0, 0);
    K230_Receive_Process();
    K230_Servo_Control(ball_target_cm);
    return;
}

float stop_ratio = 1.0f - (float)st / (float)BALL_STOP_TIME_MS;
if (stop_ratio < 0.0f) stop_ratio = 0.0f;
if (stop_ratio > 1.0f) stop_ratio = 1.0f;
stop_comp = BALL_STOP_COMP * stop_ratio;
g_line_speed = (int)((float)stop_spd * stop_ratio);
}
else
{
    stop_ready = 0;
    stop_start_ms = 0;
    stop_spd = 0;

    /* Start phase: linearly accelerate from minimum speed to cruise speed. */
    if (ramp_ready)
    {
        uint32_t et = Get_Time() - last_task_start_ms;
        if (et < BALL_START_COMP_TIME_MS)
            start_comp = BALL_START_COMP * (1.0f - (float)et /
(float)BALL_START_COMP_TIME_MS);
        if (et < BALL_RAMP_TIME_MS)
        {
            float ramp = (float)et / (float)BALL_RAMP_TIME_MS;
            if (ramp < BALL_RAMP_MIN_RATIO) ramp = BALL_RAMP_MIN_RATIO;
            if (ramp > 1.0f) ramp = 1.0f;
            g_line_speed = (int)((float)saved_spd * ramp);
        }
        else
        {
            g_line_speed = saved_spd;
        }
    }
}

/* First line tracking, then parse K230 and send compensated servo target. */
Debug_Linewalking_Run();
K230_Receive_Process();

float compensated = ball_target_cm;
compensated -= BALL_INERTIA_FWD;
if (pid_output_IRR > 0)
    compensated -= BALL_INERTIA_TURN * (float)pid_output_IRR / BALL_INERTIA_PID_MAX;
else if (pid_output_IRR < 0)
    compensated += BALL_INERTIA_TURN * (float)(-pid_output_IRR) / BALL_INERTIA_PID_MAX;
compensated += start_comp + stop_comp;

```

```
    K230_Servo_Control(compensated);  
}
```

8.4 Task 2 Steel Ball Reciprocation (questions.c)

```
// Three phases: +5cm(1.8s) -> 0cm(1.8s) -> -5cm(1.8s) -> DONE(maintain -5)  
case SUB_RUNNING: {  
    static int phase = 0;  
    static uint32_t t0 = 0;  
    if (t0 == 0) t0 = Get_Time();  
  
    float target_cm;  
    if (phase == 0) target_cm = +5.0f;  
    else if (phase == 1) target_cm = 0.0f;  
    else target_cm = -5.0f;  
  
    K230_Receive_Process();  
    K230_Servo_Control(target_cm);  
  
    if (Get_Time() - t0 >= 1800) {      // Each phase 1.8 seconds  
        t0 = Get_Time();  
        if (phase == 0) phase = 1;  
        else if (phase == 1) phase = 2;  
        else { phase = 0; Sub_State = SUB_DONE; }  
    }  
    break;  
}  
case SUB_DONE:  
    K230_Receive_Process();  
    K230_Servo_Control(-5.0f);    // keep ball at -5cm position  
    break;
```

9. Operation Procedure

1. After wiring, power on; OLED displays "K230 Init..." and waits for 4 seconds
2. Enter mode selection interface; OLED displays "Mode:1"
3. **Long press K1** to switch modes (1~5)
4. **Short press K1** to confirm and enter current mode
5. Car starts executing task; OLED displays task name + timer
6. **Short press K1 during task:** Pause; **Long press K1:** Exit back to mode selection
7. After task completion/timeout, car stops but **ball control task continues** (ball stays at target position)
8. Long press K1 to exit back to mode selection